

Ejercicio de parcial

1. Ejercicio - Bash

No, **Bash** no es una buena opción para desarrollar una aplicación web. Aunque Lucas y Martín propongan usarlo porque es una herramienta que conocen, Tomás tiene toda la razón en dudar.

Bash es un lenguaje de *scripting* diseñado principalmente para automatizar tareas del sistema operativo, manipular archivos o gestionar servidores. No está pensado para manejar solicitudes HTTP, conectar fluidamente con bases de datos relacionales, ni renderizar interfaces de usuario en el navegador. Para construir la aplicación web que plantean (que además quieren escalar con funcionalidades de estadísticas y registro de lesiones), deberían optar por tecnologías adecuadas para la web, como JavaScript/Node.js, Python, Java o PHP.

2. Ejercicio - Github

GitHub es una plataforma alojada en la nube que utiliza Git, un sistema de control de versiones.

Beneficiaría al equipo enormemente porque les permite trabajar de forma colaborativa y ordenada. Como son tres personas pensando en el sistema, GitHub les permitiría:

- Trabajar en el mismo código sin sobrescribir el trabajo del otro.
 - Mantener un historial completo de cambios (si algo se rompe, pueden volver a una versión anterior).
 - Probar nuevas funcionalidades que propuso Tomás (como el historial de partidos) en entornos aislados (ramas o *branches*) antes de unirlas al proyecto principal.
-

3. Ejercicio - SQL

- *Creamos entidades y tablas*

-- **jugadores** (id, nombre_completo, posicion_preferida, fecha_nacimiento, nacionalidad, dni)

✍ **Serial = para que la id sea autoincremental, default "argentina" por si no hay ninguna, te asigna esa como principal**

```
CREATE TABLE jugadores(  
  id SERIAL PRIMARY KEY,  
  nombre_completo VARCHAR (100) NOT NULL,  
  posicion_preferida VARCHAR (50),  
  fecha_nacimiento DATE,  
  nacionalidad VARCHAR (50) DEFAULT "Argentina",  
  dni INT,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  contrasenia VARCHAR(255) NOT NULL,  
);
```

-- **partidos** (id, fecha_hora, lugar, goles_local, goles_visitante)

```
CREATE TABLE partidos(  
  id SERIAL PRIMARY KEY,  
  fecha_hora TIMESTAMP NOT NULL,  
  lugar TEXT NOT NULL,  
  goles_local SMALLINT,  
  goles_visitante SMALLINT,  
  inscripciones_desde TIMESTAMP,  
  inscripciones_hasta TIMESTAMP NOT NULL  
);
```

-- **jugadores_partidos**

```
CREATE TABLE jugadores_partidos(  
  id_jugador INT REFERENCES(jugadores),  
  id_partido INT REFERENCES(partidos),  
  es_local BOOL NOT NULL,  
  goles_ anotados SMALLINT NOT NULL DEFAULT 0,  
  asistencias_hechas SMALLINT NOT NULL DEFAULT 0,  
  PRIMARY KEY (id_jugador, id_partido)  
);
```

-- **inscripciones** (partido, jugador, fecha_inscripcion)

```
CREATE TABLE inscripciones(  
  id_jugador INT REFERENCES(jugadores),  
  id_partido INT REFERENCES(partidos),  
  fecha_inscripcion TIMESTAMP NOT NULL,  
  PRIMARY KEY (id_jugador, id_partido)  
);
```

- **Justificaciones**

- Al ser un grupo de amigos, normalmente considero que normalmente no hay equipos fijos, por lo tanto, los jugadores van a ir mezclandose.
 - Tendriamos una tabla para los jugadores, la cual contendría campos como una ID autoincremental, el nombre del jugador, su posicion preferida, su fecha de nacimiento, su email y contraseña para iniciar sesión en el sistema. La idea es que cada persona del grupo se registre y complete sus datos.
 - Tambien tendriamos una tabla para los partidos, con los campos de la fecha y hora del mismo, a ubicacion, tambien un ID autoincremental, la cantidad de goles del equipo local y visitante.
 - Otra tabla es la de inscripciones, la cual contiene una FK hacia un jugador y otra hacia un partido junto con la fecha de inscripción del jugador. Esta tabla nos sirve para registrar que un jugador se anoto para jugar un partido en particular. La primary key es la tupla (id_jugador, id_partido).
 - Por ultimo, una tabla jugadores_partido que nos servira para indicar que jugadores jugaron que partidos. Esta tabla representa una relacion muchos a muchos. Tendra las mismas FK que la tabla anterior, un campo booleano "es_local", indicando si el jugador jugo para el equipo local y los goles anotados y asistencias hechas por el jugador.
 - Las entidades principales serian los jugadores y los partidos.
 - Las relaciones que tenemos son desde jugadores partidos e inscripciones hacia la tabla de jugadores y la tabla de los partidos. Las FK sirven para referenciar la PRIMARY KEY DE OTRA TABLA, en este caso la de jugadores.
-

4. Ejercicio - SQL (sentencia)

GROUP BY NO ENTRA

- Goles hechos por partido (fecha) por cada jugador (nombre completo)

(Para mostrar = **select**)

```
SELECT p.fecha_hora, j.nombre_completo, jp.goles_anotados
FROM jugadores_partidos jp
JOIN jugadores ON jp.id_jugador = j.id
JOIN partidos p ON jp.id_partido = p.id
WHERE j.p goles_anotados distinto de = 0
```

```
ORDER BY p.fecha_hora, jp.goles_ anotados DESC, jp.asistencias_hechas DESC,  
j.nombre_completo;
```

-- Se muestra, para cada partido, su fecha y la cantidad de goles anotados por cada jugador (con su nombre), ordenada primero por fecha del partido (de mas viejo a mas nuevo) y luego por cantidad de goles hechos (de mayor a menor). En caso de empate de goles, se define por cantidad de asistencias. Si aun hubiera empate, se define por orden alfabetico del nombre del jugador. Solo se muestran los jugadores que marcaron al menos un gol en cada partido.

CASO MULTIPLE FROM

```
SELECT p.fecha_hora, j.nombre_completo, jp.goles_ anotados  
FROM jugadores_partidos jp, jugadores j, partidos p  
WHERE jp.id_jugador = j.id AND jp.id_partido = p.id AND jp.goles_ anotados  
distinto de = 0  
ORDER BY p.fecha_hora, jp.goles_ anotados DESC, jp.asistencias_hechas DESC,  
j.nombre_completo;
```

5. Ejercicio - Ingenieria de Software

Hasta el momento, el equipo ha aplicado principalmente la etapa de **Análisis de Requerimientos** (o Ingeniería de Requisitos).

Se han dedicado a entender y definir el problema (las injusticias y desorganización de WhatsApp) y a estipular qué debe hacer el sistema para solucionarlo (una aplicación web de registro ordenado). Además, Tomás ha hecho una labor de *levantamiento de requerimientos a futuro* al proponer las estadísticas y la escalabilidad de la plataforma. También están rozando la etapa de **Diseño/Planificación**, al debatir qué arquitectura técnica (web vs. móvil) y herramientas (Bash vs. otras) utilizarán.

6. Ejercicio - Docker

Sí, absolutamente. Aunque los tres tengan computadoras con Linux, es muy probable que tengan diferentes configuraciones, versiones de librerías o bases de datos instaladas.

Docker les permitiría empaquetar la aplicación web y su base de datos en "contenedores" aislados y estandarizados. Esto asegura que el entorno de ejecución sea idéntico para los tres, eliminando el clásico problema de "en mi máquina funciona, pero en la tuya no". Además, cuando decidan subir su aplicación web a un servidor en internet, Docker hará que el despliegue sea muchísimo más fácil y seguro.

7. Ejercicio - Regex (expresiones regulares)

- Tomas recolectó un montón de equipos que podrían usar el sistema y los guardó en un archivo csv.
- El formato del archivo es NombreEquipo;PartidosPorSemana;TeléfonoContacto;MailContacto-
- Quieren arrancar contactando a quienes jueguen 1 vez por semana y el Mail sea un gmail.
- Escribir una expresión regular que liste los equipos a contactar.

```
^[^;]+;1;[^;]+;[^@]+gmail\.com$
```

- `^` : Indica el **inicio** de la línea. Asegura que la evaluación empiece desde el primer carácter.
- `[^;]+` : Corresponde al **NombreEquipo**.
 - Los corchetes `[^;]` significan "cualquier carácter que **NO** sea un punto y coma".
 - El `+` significa "uno o más de estos caracteres". Básicamente lee el nombre hasta chocar con el primer punto y coma.
- `;` : Lee el primer punto y coma separador literal.
- `1` : Corresponde a **PartidosPorSemana**. Obliga a que el valor sea estrictamente un `1`. Si alguien juega 2 veces, la expresión lo descarta automáticamente.
- `;` : El segundo punto y coma separador.
- `[^;]+` : Corresponde al **TeléfonoContacto**. Igual que al principio, lee cualquier cosa (números, espacios, guiones) hasta chocar con el siguiente punto y coma.
- `;` : El tercer y último punto y coma separador.
- `[^@]+` : Aquí empieza a leer el **MailContacto**. Busca uno o más caracteres que **NO** sean una arroba (`@`). Es decir, está leyendo el nombre de usuario del correo (ej: "losgoleadores").
- `gmail\.com` : Busca exactamente el texto "gmail.com". Se usa la barra invertida `\` antes del punto (`\.`) para "escaparlo", ya que en Regex un punto suelto significa "cualquier carácter", pero aquí queremos que busque un punto real.
- `$` : Indica el **fin** de la línea. Asegura que no haya texto basura después del ".com".